

POC Assignment: E-Commerce Order Management & Search Service

A practical proof-of-concept integrating MongoDB Community Edition with OpenSearch.

Your Objective

Build a miniature e-commerce system where MongoDB Community Edition is the operational source of truth and order documents are synchronized to OpenSearch for a fast admin search dashboard.

Selected NoSQL Stack

Primary database: MongoDB Community Edition. It is free to self-host and provides a document model, single-document atomicity, multi-document transactions, replica sets, indexes, and change streams. This POC uses embedded order items for fast aggregate reads while retaining separate user and product collections.

Search index: OpenSearch. It is an Apache 2.0-licensed search and analytics engine supporting full-text queries, filters, facets, and aggregations.

Licensing note: MongoDB Community Edition uses the Server Side Public License (SSPL). It is free and source-available, but the SSPL is not an OSI-approved open-source licence.

Product Specifications: Screen-by-Screen Breakdown

Implement the following screens. The **Data Source** tag dictates whether the backend reads from MongoDB or from OpenSearch.

Data Source: MongoDB Screen 1: Customer Storefront

Purpose: A simple customer-facing view where a user can browse available products and add them to a mock cart.

UI Components Required:

- User Identifier:** A dropdown to “Log In As” one of the seeded test users, such as John Doe or Jane Smith. No real authentication is required.
- Product Grid:** Displays active documents from the products collection. Show title, description, current price, and available stock.
- Add to Cart Button:** Accumulates items in local state before checkout.

Developer Action: Implement GET /api/products. Query MongoDB directly with a projection that returns only the required storefront fields. Do not read products from OpenSearch.

Data Source: MongoDB Screen 2: Checkout & Order Placement

Purpose: The transactional boundary where the order is validated, inventory is reserved, and the order is officially recorded.

UI Components Required:

- Order Summary:** List of items in the cart and the calculated total_amount.
- “Place Order” Button:** Triggers the core checkout logic.
- Success/Failure Notification:** Clearly displays whether the order was accepted, rejected for insufficient stock, or failed.

Developer Action: Implement POST /api/orders. Never trust prices or totals from the browser. The server must reload products, validate availability, calculate totals, and use a MongoDB transaction to:

- Conditionally decrement stock only where sufficient inventory exists.
- Insert one order document containing embedded customer and product snapshots.
- If using the outbox strategy, insert the corresponding event into outbox_events.

If any operation fails, abort the entire transaction. Require an idempotency key and enforce it with a unique index so retries cannot create duplicate orders. After commit, synchronize the order to OpenSearch using one of the required strategies.

Data Source: OpenSearch Screen 3: Admin Search Dashboard (Core Feature)

Purpose: A back-office tool for staff to locate orders quickly using partial text, customer names, or line items.

UI Components Required:

- Omni-Search Bar:** A single input that executes a multi-field query. Searching for “Wireless” should return orders containing “Wireless Mouse” and orders placed by “Wendy Wireless”.
- Faceted Filters (Sidebar):**
 - Order status: Pending, Processing, Shipped, Cancelled
 - Date range: From/To date pickers
 - Total amount: Minimum/maximum range
- Results Data Table:** Displays Order ID, Date, Customer Name, Total Amount, and Status. Results should update as filters change.
- KPI Dashboard:** Cards populated from OpenSearch aggregations:
 - Total Revenue of Filtered Orders
 - Count of Orders by Status

Developer Action: Implement POST /api/search/orders. Translate the UI state into OpenSearch Query DSL using bool, multi_match, range, and terms/metric aggregations. **Never query MongoDB for this screen.**

Data Source: Hybrid Screen 4: Admin Order Details

Purpose: The canonical detailed view of a specific order, opened by clicking an Order ID in the search dashboard.

UI Components Required:

- Order Meta:** ID, timestamp, customer snapshot, and current version.
- Itemized Receipt:** Full breakdown of the embedded items array.
- Status Toggle:** A dropdown to update the order status.
- Sync Status Indicator (Recommended):** Shows whether the latest MongoDB order version is indexed in OpenSearch.

Developer Actions:

- Read:** Implement GET /api/orders/:id. Read the source-of-truth document from MongoDB.
- Update:** Implement PATCH /api/orders/:id/status. Use optimistic concurrency by filtering on both _id and the supplied version, then increment version atomically. Return HTTP 409 if another update has already changed the document.

Document & Index Architecture

Phase 1: MongoDB Collections (Source of Truth)

Create one MongoDB database named ecommerce with these collections:

- users — one document per test user
- products — one document per product, including current price and stock
- orders — one document per order, with customer and item snapshots embedded
- outbox_events — required only when implementing the transactional outbox strategy

You may use ObjectId, UUIDs, or readable string identifiers consistently. An order document should resemble:

```
{
  "_id": "order_1024",
  "order_date": "2026-07-10T10:30:00Z",
  "updated_at": "2026-07-10T10:30:00Z",
  "version": 1,
  "status": "PROCESSING",
  "currency": "USD",
  "total_minor": 15050,
  "idempotency_key": "checkout-7cc4f1",
  "customer": {
    "id": "user_5",
    "name": "John Doe",
    "email": "john@example.com"
  },
  "items": [
    {
      "product_id": "product_42",
      "sku": "MOUSE-42",
      "title": "Wireless Mouse",
      "quantity": 1,
      "unit_price_minor": 5016,
      "line_total_minor": 5016
    },
    {
      "product_id": "product_88",
      "sku": "KEYBOARD-88",
      "title": "Mechanical Keyboard",
      "quantity": 1,
      "unit_price_minor": 10034,
      "line_total_minor": 10034
    }
  ]
}
```

Modelling rule: Customer names, product titles, SKUs, and charged prices are intentional snapshots. Historical orders must not change when a user edits a profile or a product is renamed or repriced later. Store money in integer minor units to avoid floating-point errors.

Create at least the following MongoDB indexes:

```
db.products.createIndex({ active: 1, title: 1 })
db.products.createIndex({ sku: 1 }, { unique: true })
db.orders.createIndex({ idempotency_key: 1 }, { unique: true })
db.orders.createIndex({ status: 1, order_date: -1 })
db.orders.createIndex({ "customer.id": 1, order_date: -1 })
db.outbox_events.createIndex({ processed_at: 1, created_at: 1 })
```

The final outbox index is needed only if that strategy is implemented. Use explain("executionStats") to demonstrate that the key operational queries use appropriate indexes. Do not reproduce the complete full-text admin dashboard in MongoDB.

Phase 2: OpenSearch Index

Create an index named orders. Use the stable MongoDB order ID as the OpenSearch document _id. Include the application version so sync freshness and event ordering can be verified:

```
{
  "order_id": "order_1024",
  "source_version": 1,
  "order_date": "2026-07-10T10:30:00Z",
  "updated_at": "2026-07-10T10:30:00Z",
  "status": "PROCESSING",
  "currency": "USD",
  "total_minor": 15050,
  "customer": {
    "id": "user_5",
    "name": "John Doe",
    "email": "john@example.com"
  },
  "items": [
    { "product_id": "product_42", "sku": "MOUSE-42", "title": "Wireless Mouse", "quantity": 1 },
    { "product_id": "product_88", "sku": "KEYBOARD-88", "title": "Mechanical Keyboard", "quantity": 1 }
  ]
}
```

Map exact filter fields such as status, currency, IDs, and SKUs as keyword; names and titles as text with keyword subfields; dates as date; and totals and versions as numeric types.

The Synchronization Challenge

Implement and compare at least **two** strategies. Every strategy must be idempotent, retryable, and able to rebuild OpenSearch from MongoDB.

- Option A — Application Dual-Write:** Commit the MongoDB transaction, then index the order in OpenSearch. If OpenSearch fails, the order remains valid and a retry queue or reconciliation process must repair the index.
- Option B — MongoDB Change Stream (Recommended):** Run MongoDB Community Edition as a replica set. A worker watches the orders collection, transforms changes, and upserts documents into OpenSearch. Persist the resume token only after successful indexing and resume from it after a restart.
- Option C — Transactional Outbox:** Insert an outbox event in the same MongoDB transaction as the order mutation. A worker claims unprocessed events, indexes them, and marks them processed. Define retry and dead-letter behaviour.
- Option D — Periodic Reconciliation:** A scheduled worker scans orders changed since a checkpoint and performs bulk upserts. Also provide a full reindex command.

Index using the stable order ID so retries overwrite the same OpenSearch document. Reject stale events by comparing source_version. MongoDB is authoritative whenever the two systems disagree.

Technical Acceptance Criteria

- All services run locally using Docker Compose: application, MongoDB Community Edition, OpenSearch, and the synchronization worker.
- MongoDB runs as at least a single-node replica set in local development so transactions and change streams can be exercised.
- Seed data includes at least 5 users, 20 products, and 100 orders with varied dates and statuses.
- Checkout atomically reserves stock and creates the order; an insufficient-stock item rolls back the complete transaction.
- Duplicate requests with the same idempotency key return the original result and do not create duplicate orders or decrement stock twice.
- Concurrent status updates demonstrate optimistic concurrency; stale versions receive HTTP 409.
- Search supports text, status, date, and amount filters plus revenue and status-count aggregations.
- OpenSearch downtime does not prevent successful order placement; synchronization catches up after recovery.
- A full reindex procedure can rebuild OpenSearch entirely from MongoDB.
- Automated tests cover order creation, total calculation, transaction rollback, idempotency, concurrency, query translation, and sync retries.

Learning Milestones

Before submitting, ensure you can confidently answer:

- Aggregate design:** Why are order items embedded while users and products remain separate collections?
- Atomicity:** Which work is covered by single-document atomicity, and why does inventory reservation require a multi-document transaction?
- Concurrency:** How does the version filter prevent lost updates?
- Change streams:** Why is a replica set required, what is a resume token, and when can recovery require reconciliation?
- Split-brain:** What happens if OpenSearch is unavailable for five minutes while orders continue to be placed?
- Search modelling:** Why are order fields duplicated into OpenSearch instead of performing cross-system lookups at query time?
- Consistency:** Which screens read the authoritative state and which are eventually consistent?

Submission Deliverables

- Source code and a concise README.md with setup, run, and test instructions
- docker-compose.yml, replica-set initialization, and seed-data scripts
- MongoDB schema-validation rules and index creation scripts
- OpenSearch index mapping and sample search requests
- A short comparison of the two synchronization strategies implemented
- Automated test results and a demo showing recovery after OpenSearch downtime